CONSTRAINT-BASED PROBLEM SOLVING

University Final Examination Scheduling Using CSP

Problem Description

A university must schedule five final examinations—Artificial Intelligence (A), Database Systems (B), Computer Networks (C), Operating Systems (D), and Software Engineering (E)—into three slots: Slot1, Slot2, and Slot3. Conflicting courses share students and therefore must be assigned different slots.

Conflicting pairs: A–B, A–C, B–C, B–D, C–D, C–E, and D–E.

(a) PEAS Description

PEAS component	Description
Performance measure	Assign every course a slot; satisfy all conflict constraints; avoid clashes; produce a valid timetable with minimal search effort.
Environment	Five courses (A–E), three exam slots, student-enrollment overlaps, and the listed conflict constraints.
Actuators	Assign or revise a course's slot; maintain and output the timetable.
Sensors	Read course data, available slots, enrollment-conflict information, current assignments, and domain/constraint status.

(b) Type of Intelligent Agent

A goal-based agent is best suited to this task. It searches for an assignment that reaches the goal state: every course has one slot and no conflicting pair shares a slot. It can compare candidate assignments against the constraints and backtrack when a choice prevents completion. A simple reflex agent lacks sufficient search over future consequences; a model-based agent can represent the state but does not, by itself, specify the goal-directed search; and a utility-based agent is unnecessary because any conflict-free timetable is acceptable and no preference among valid timetables is given.

(c) Constraint Satisfaction Problem (CSP) Model

Variables: A, B, C, D, E, one variable for each course.

Domains: D(A) = D(B) = D(C) = D(D) = D(E) = {Slot1, Slot2, Slot3}.

Constraints: Each listed conflicting pair must have different values: $A \neq B$, $A \neq C$, $B \neq C$, $B \neq D$, $C \neq D$, $C \neq E$, and $D \neq E$.

Constraint graph: Vertices are {A, B, C, D, E}. An undirected edge joins each pair that must have different slots.

Vertex	Adjacent vertices
A	B, C
B	A, C, D
C	A, B, D, E
D	B, C, E
E	C, D

The graph edges are: {(A,B), (A,C), (B,C), (B,D), (C,D), (C,E), (D,E)}.

(d) Plain Backtracking Search

Use the fixed variable order A → B → C → D → E and try Slot1 → Slot2 → Slot3 for each variable. An assignment attempt is counted whenever a slot is tried for a variable. A rejected attempt violates a constraint with an already assigned variable. The trace below records each attempt, including assignments that are later undone.

Step	Variable	Tried value	Outcome	State / action
1	A	Slot1	Success	A = Slot1
2	B	Slot1	Fail: A–B	Backtrack from B; try next value
3	B	Slot2	Success	A = Slot1, B = Slot2
4	C	Slot1	Fail: A–C	Try next value for C
5	C	Slot2	Fail: B–C	Try next value for C
6	C	Slot3	Success	A = Slot1, B = Slot2, C = Slot3
7	D	Slot1	Success	D differs from B and C
8	E	Slot1	Success	E differs from C and D

Final timetable from plain backtracking:

Course	Exam slot
A — Artificial Intelligence	Slot1
B — Database Systems	Slot2
C — Computer Networks	Slot3
D — Operating Systems	Slot1
E — Software Engineering	Slot2

The search makes 8 assignment attempts and 0 chronological backtracks (there are failed value attempts, but no previously successful variable assignment needs to be undone).

(e) Backtracking Search with Forward Checking

Forward checking removes from each unassigned neighboring variable's domain any value that conflicts with the newly assigned variable. If any unassigned domain becomes empty, the algorithm backtracks immediately. The same fixed variable and value order is used.

Step	Assignment	Domains after forward checking	Action
1	Assign A = Slot1	B: {Slot2, Slot3}; C: {Slot2, Slot3}; D: {Slot1, Slot2, Slot3}; E: {Slot1, Slot2, Slot3}	Continue
2	Try B = Slot2	C: {Slot3} after removing Slot1 (A–C) and Slot2 (B–C); D: {Slot1, Slot3} after	Continue

		removing Slot2 (B–D); E unchanged	
3	Assign C = Slot3	D: {Slot1} after removing Slot3 (C–D); E: {Slot1, Slot2} after removing Slot3 (C–E)	Continue
4	Assign D = Slot1	E: {Slot2} after removing Slot1 (D–E)	Continue
5	Assign E = Slot2	All variables assigned; all constraints satisfied	Success

Note: At step 2, B = Slot1 is tried first and rejected because A–B is a conflict. Then B = Slot2 is assigned. Forward checking removes Slot1 from C due to A–C and Slot2 from C due to B–C, leaving C = {Slot3}.

Final timetable from forward checking:

Course	**Exam slot**
A — Artificial Intelligence	Slot1
B — Database Systems	Slot2
C — Computer Networks	Slot3
D — Operating Systems	Slot1
E — Software Engineering	Slot2

Forward-checking assignment attempts: 6 (A=Slot1; B=Slot1 rejected; B=Slot2; C=Slot3; D=Slot1; E=Slot2). There are 0 chronological backtracks because the first consistent path reaches a solution. Domain pruning prevents the search from trying values already known to be impossible for future variables.

(f) Comparison and Explanation

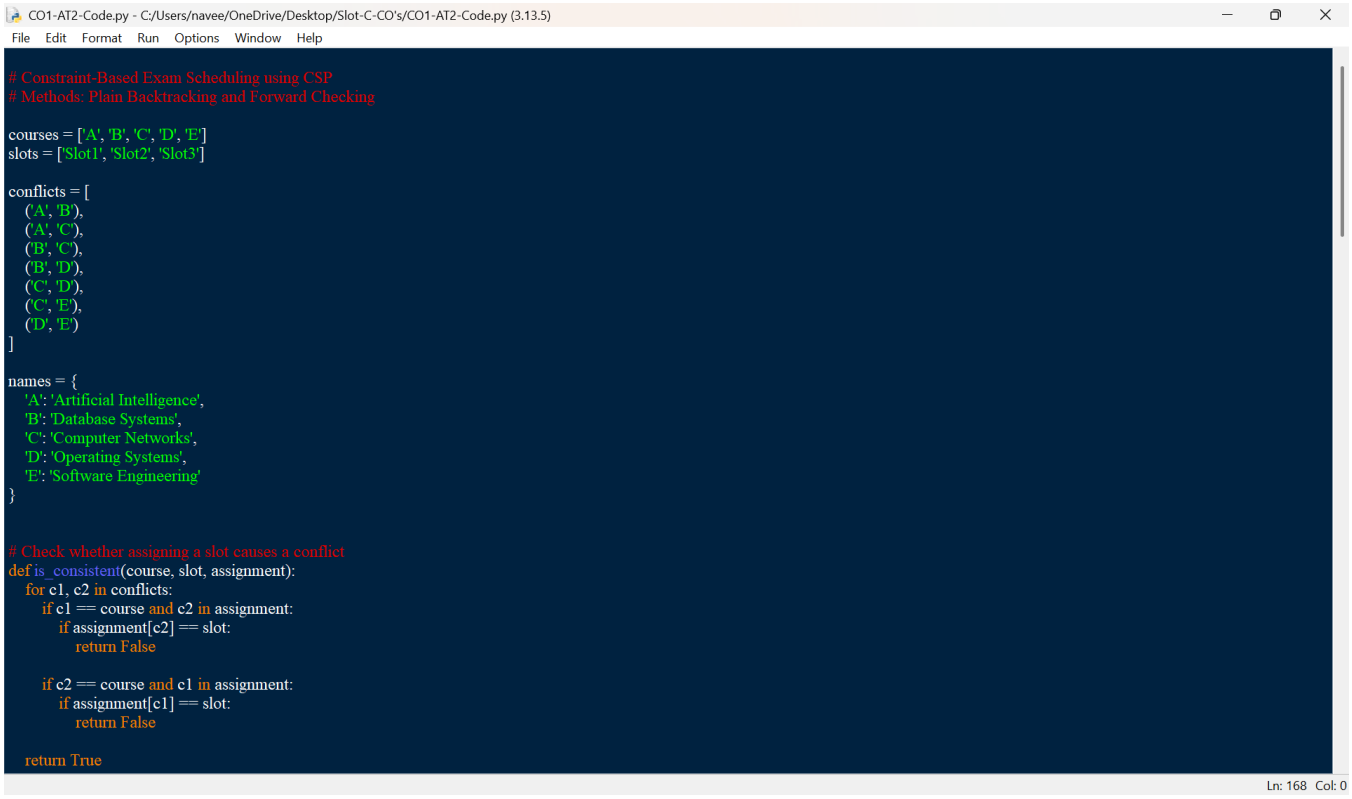
Measure	**Plain backtracking**	**Backtracking + forward checking**
Assignment attempts	8	6
Failed value attempts	3 (B=Slot1, C=Slot1, C=Slot2)	1 (B=Slot1); the remaining failures are eliminated by pruning
Chronological backtracks	0	0
Domain pruning	No explicit pruning of future domains	Removes values from unassigned neighbors after each assignment
Search effort	More attempts are needed to discover conflicts when a variable is reached	Lower effort because future conflicts are detected earlier

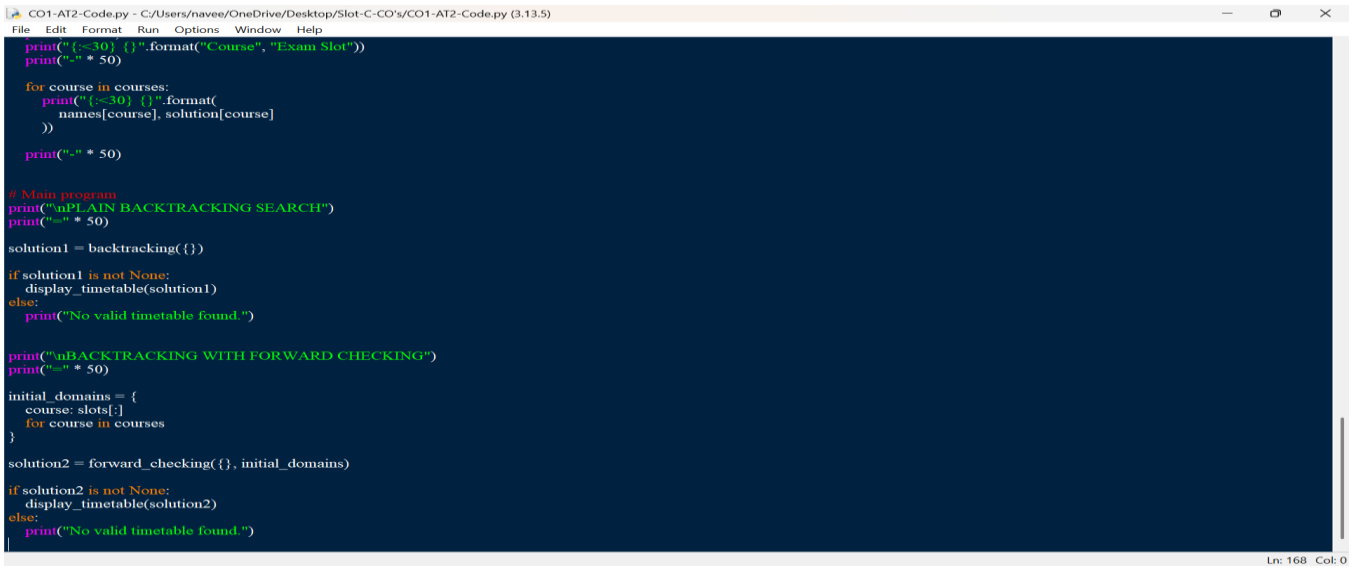
Why forward checking reduces search effort: plain backtracking checks a variable's candidate value only when that variable is selected. Forward checking propagates the effect of each assignment to the domains of unassigned neighboring variables. For example, after A = Slot1 and B = Slot2, C is immediately restricted to {Slot3}; the algorithm does not need to test C = Slot1 and C = Slot2 separately. This early pruning reduces assignment attempts for this instance.

Conclusion: Both methods produce a valid conflict-free timetable. With the specified ordering, plain backtracking makes 8 assignment attempts, while forward checking makes 6. Neither method needs a

chronological backtrack on this particular search path, but forward checking reduces the number of explicit failed attempts by detecting impossible values in advance.

Programming:





```
CO1-AT2-Code.py - C:/Users/navee/OneDrive/Desktop/Slot-C-CO's/CO1-AT2-Code.py (3.13.5)
File  Edit  Format  Run  Options  Window  Help
# Method 1: Plain Backtracking
def backtracking(assignment):
    if len(assignment) == len(courses):
        return assignment.copy()

    course = courses[len(assignment)]

    for slot in slots:
        print("Trying", course, "=", slot)

        if is_consistent(course, slot, assignment):
            assignment[course] = slot

            result = backtracking(assignment)

            if result is not None:
                return result

            print("Backtracking from", course, "=", slot)
            del assignment[course]

        else:
            print("Conflict detected!")

    return None


# Method 2: Backtracking with Forward Checking
def forward_checking(assignment, domains):
    if len(assignment) == len(courses):
        return assignment.copy()

    course = courses[len(assignment)]

    for slot in slots:
        if slot not in domains[course]:
            continue
```

Ln: 168 Col: 0

Output:

```
IDLE Shell 3.13.5
File  Edit  Shell  Debug  Options  Window  Help
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
====== RESTART: C:/Users/navee/OneDrive/Desktop/Slot-C-CO's/CO1-AT2-Code.py ======

PLAIN BACKTRACKING SEARCH
==================================================
Trying A = Slot1
Trying B = Slot1
Conflict detected!
Trying B = Slot2
Trying C = Slot1
Conflict detected!
Trying C = Slot2
Conflict detected!
Trying C = Slot3
Trying D = Slot1
Trying E = Slot1
Conflict detected!
Trying E = Slot2
```

```
IDLE Shell 3.13.5
File  Edit  Shell  Debug  Options  Window  Help
Operating Systems          Slot1
Software Engineering       Slot2
--------------------------------------------------

BACKTRACKING WITH FORWARD CHECKING
==================================================
Trying A = Slot1
Domains after assigning A = Slot1
B : ['Slot2', 'Slot3']
C : ['Slot2', 'Slot3']
D : ['Slot1', 'Slot2', 'Slot3']
E : ['Slot1', 'Slot2', 'Slot3']
Trying B = Slot2
Domains after assigning B = Slot2
C : ['Slot3']
D : ['Slot1', 'Slot3']
E : ['Slot1', 'Slot2', 'Slot3']
Trying C = Slot3
Domains after assigning C = Slot3
D : ['Slot1']
E : ['Slot1', 'Slot2']
Trying D = Slot1
Domains after assigning D = Slot1
E : ['Slot2']
Trying E = Slot2
Domains after assigning E = Slot2

Final Examination Timetable
--------------------------------------------------
Course                     Exam Slot
--------------------------------------------------
Artificial Intelligence    Slot1
Database Systems           Slot2
Computer Networks          Slot3
Operating Systems          Slot1
Software Engineering       Slot2
--------------------------------------------------
>>>
```

Ln: 66 Col: 0

Result:

Both algorithms find a valid timetable satisfying all seven conflict constraints. Forward checking reduces unnecessary search by removing unavailable slots from the domains of unassigned courses.